

Laboratory of Decision Support Systems

Barbieri Bruno

Braghin Federica

Vannucci Nicholas

December 2025

Abstract

This is a concise abstract summarizing the motivation, method, and main results of the work.

1 Introduction

Introduce the problem, context, and contributions.

2 Related Work

Brief overview of relevant literature.

3 Methodology

Describe the proposed method or theoretical framework.

3.1 Assignment 4 – Data Warehouse Schema Design

L'Assignment 4 richiedeva il passaggio da un database operativo a un data warehouse orientato all'analisi, con l'obiettivo di progettare uno schema multidimensionale in grado di analizzare il numero di stream registrati a un mese dalla pubblicazione di una canzone (*Streams1Month*).

Lo schema fornito nella consegna era indicato come riferimento non vincolante, lasciando libertà progettuale nella definizione della granularità, delle dimensioni e delle relazioni. Seguendo l'approccio discusso a lezione, la progettazione è partita dall'analisi delle *business questions* e delle query che si intendevano supportare.

Da tale analisi è emerso che la misura centrale del data warehouse è unica:

- **Streams1Month**, ovvero il numero di stream registrati a un mese dalla pubblicazione del brano.

A partire da questa misura sono state individuate le dimensioni rilevanti per l'analisi:

- **DimSong**, che rappresenta la canzone pubblicata;
- **DimArtist**, che rappresenta gli artisti che hanno partecipato alla canzone;
- **DimDate**, che rappresenta la data di pubblicazione del brano;
- **DimArtistGeography**, che rappresenta il luogo di nascita dell'artista;
- **DimLyrics**, che rappresenta il testo del brano e le sue componenti;

- **DimSymphony**, che rappresenta le caratteristiche acustiche del brano;
- **DimAlbum**, che rappresenta il contesto editoriale della canzone, non necessariamente coincidente con un album tradizionale.

Un'ulteriore scelta progettuale rilevante è stata quella di non utilizzare le *natural key* presenti nei dataset originali, in quanto fortemente duplicate e poco affidabili. Sono state pertanto introdotte chiavi surrogate univoche, generate tramite UUID, al fine di garantire coerenza, stabilità e corretta identificazione delle dimensioni.

Evoluzione dello schema e scelte progettuali

In una prima fase progettuale è stata considerata una fact table denominata *PublishedSong*, con granularità definita come *una riga per canzone pubblicata da un artista*, collegata alle dimensioni Song, Artist, Date, Lyrics e Symphony.

In tale schema, la gestione degli artisti in featuring sarebbe stata possibile esclusivamente tramite l'introduzione di una *bridge table* tra *DimSong* e *DimArtist*. Tuttavia, questo approccio presentava alcune criticità rilevanti. In primo luogo, l'uso della bridge table avrebbe condotto a una struttura che non aderiva pienamente né allo *star schema* né allo *snowflake schema*, aumentando il numero di join necessari per l'esecuzione delle query analitiche e riducendo l'efficienza complessiva del sistema.

In secondo luogo, la gestione dei featuring risultava meno naturale dal punto di vista analitico, poiché il ruolo dell'artista (artista primario o featuring) non era direttamente associato alla misura analizzata, ma derivato da una struttura intermedia.

Sono state inoltre considerate altre alternative progettuali, tra cui:

- l'introduzione di un attributo *Role* nella bridge table;
- l'utilizzo di una *degenerate dimension* per distinguere tra artisti primari e featuring;
- la creazione di una dimensione di gruppo di artisti tra la fact table e la dimensione Artist.

Sebbene concettualmente corrette, tali soluzioni introducevano una maggiore complessità strutturale o rendevano meno immediate le analisi per artista, risultando meno adatte agli obiettivi dell'assignment.

Schema finale del Data Warehouse

La soluzione finale adottata è stata la creazione di una fact table denominata *FactParticipation*, che modella esplicitamente la partecipazione di un artista a una canzone.

La granularità finale è definita come:

una riga = partecipazione di un artista a una canzone in una determinata data.

Di conseguenza, la stessa canzone può comparire più volte all'interno della fact table, una volta per ciascun artista coinvolto. Questo comportamento è atteso ed è il risultato di una granularità più fine, scelta in modo consapevole per rappresentare correttamente la relazione molti-a-molti tra artisti e canzoni, evitando duplicazioni semantiche delle misure.

Il ruolo dell'artista è rappresentato tramite l'attributo *IsPrimary*, modellato come *degenerate dimension* all'interno della fact table:

- *IsPrimary* = 1 identifica l'artista principale;
- *IsPrimary* = 0 identifica gli artisti in featuring.

La fact table è collegata direttamente alle dimensioni *DimSong*, *DimArtist* e *DimDate*, mentre le dimensioni Song e Artist generano strutture a *snowflake schema* rispettivamente verso *DimAlbum*, *DimLyrics*, *DimSymphony* e *DimArtistGeography*. L'insieme delle foreign key identifica univocamente i record della fact table, in accordo con quanto discusso a lezione.

La dimensione *DimDate* è stata collegata direttamente alla fact table, in quanto rappresenta il riferimento temporale dell'evento analizzato (stream a un mese dalla pubblicazione). Altre date semanticamente differenti, come la data di rilascio dell'album, sono state mantenute come attributi all'interno delle rispettive dimensioni, evitando ambiguità e join aggiuntivi.

Implementazione

Una volta definito lo schema multidimensionale del Data Warehouse, è stata realizzata anche la sua implementazione fisica sul database server utilizzando SQL Server Management Studio. In particolare, è stato scritto ed eseguito uno script SQL per la creazione delle tabelle dimensionali e della fact table, in piena coerenza con lo schema logico descritto.

Le tabelle sono state inizialmente create vuote, definendo esplicitamente:

- gli attributi e i relativi tipi di dato;
- le chiavi primarie;
- le chiavi esterne che implementano le relazioni tra fact table e dimensioni.

Le foreign key riflettono correttamente la struttura del Data Warehouse, garantendo allineamento tra modello concettuale, logico e implementazione fisica. Sono stati inoltre introdotti vincoli e scelte sui tipi di dato per garantire coerenza e qualità dei dati. In particolare:

- l'attributo *Gender* della dimensione *DimArtist* è stato vincolato ai soli valori ammessi (*M* o *F*) tramite una *check constraint*;
- i tipi di dato numerici sono stati scelti in base alla natura degli attributi, utilizzando *INT* per conteggi discreti, *FLOAT* per misure continue e *BIT* per attributi booleani come *IsPrimary*;
- la misura *Streams1Month* è stata definita come attributo numerico adeguato a rappresentare valori di conteggio di grandi dimensioni.

Questa fase ha permesso di separare in modo chiaro la progettazione dello schema dalla fase di popolamento dei dati, che viene affrontata negli assignment successivi. Il caricamento effettivo dei dati nel Data Warehouse è infatti realizzato nell'Assignment 6, tramite strumenti ETL, a partire dai file CSV prodotti nell'Assignment 5.

3.2 Assignment 5 – Data Preparation and Dataset Splitting

L'Assignment 5 richiedeva lo sviluppo di un programma Python in grado di suddividere i dati di input in file distinti, uno per ciascuna dimensione e per la fact table del Data Warehouse progettato nell'Assignment 4, al fine di preparare i dataset per le successive fasi di caricamento.

Per soddisfare tale richiesta, il processo di *data preparation* è stato articolato in due fasi consecutive e complementari:

- pulizia e normalizzazione dei dati;
- suddivisione ed esportazione dei dataset, uno per ciascuna tabella del Data Warehouse.

Pulizia e normalizzazione dei dati

La prima fase è stata implementata tramite lo script `prepare_dw_datasets.py`, il cui obiettivo è rendere i dati coerenti con lo schema multidimensionale progettato.

Lo script carica i dataset in formato JSON dalla directory `dataset/correct_ids` ed esegue una serie di operazioni di pulizia e normalizzazione prima dello *split* vero e proprio. In particolare, vengono effettuate le seguenti operazioni:

- uniformazione dei tipi di dato (interi, float e stringhe);
- gestione consistente dei valori mancanti e non validi;
- normalizzazione degli attributi testuali;
- correzione di incongruenze sintattiche ereditate dai dataset originali;
- generazione e validazione delle chiavi surrogate, in linea con lo schema del Data Warehouse;
- normalizzazione della dimensione temporale e derivazione dell'attributo *Season*.

Il risultato di questa fase è un insieme di dataset puliti e coerenti, salvati nella directory `dataset/cleaned_json`, che rappresentano una versione intermedia pronta per essere suddivisa secondo le tabelle del Data Warehouse.

Esportazione

La seconda fase è stata implementata tramite lo script `export_dw_csv.py`, che realizza esplicitamente quanto richiesto dalla consegna, ovvero la suddivisione dei dati in file separati, uno per ciascuna tabella del Data Warehouse.

A partire dai dataset normalizzati, lo script genera un file CSV per ogni dimensione e per la fact table. Ogni file CSV contiene esclusivamente gli attributi previsti dallo schema del Data Warehouse ed è costruito in modo da rispettarne la granularità.

In particolare, la fact table *FactParticipation* mantiene la granularità definita nell'Assignment 4, ovvero:

una riga per partecipazione artista–canzone.

Essa include la misura *Streams1Month* e l'attributo *IsPrimary*, modellato come *degenerate dimension*.

I file CSV prodotti costituiscono quindi l'output finale dell'Assignment 5 e rappresentano l'input diretto per le successive fasi di caricamento del Data Warehouse.

L'Assignment 5 ha permesso di trasformare i dati di partenza in un insieme di file strutturati, uno per ciascuna tabella del Data Warehouse, come richiesto dalla consegna. La separazione tra fase di normalizzazione e fase di esportazione consente di ottenere dati puliti, coerenti e direttamente utilizzabili, facilitando le successive attività di popolamento e analisi.

3.3 Assignment 6 – Data Uploading with Python

L'Assignment 6 richiedeva lo sviluppo di un programma Python in grado di popolare il database del Data Warehouse (*Group_ID_8_DB*), rispettando lo schema e le relazioni definite nell'Assignment 4 e utilizzando i dati preparati negli assignment precedenti.

Poiché il caricamento completo dei dati può risultare oneroso in termini di tempo e risorse, è stato richiesto di progettare il codice in modo appropriato, tenendo conto sia delle prestazioni sia della stabilità del processo.

Il caricamento dei dati è stato implementato tramite uno script Python che utilizza la libreria `pyodbc` per stabilire una connessione a *SQL Server*. La connessione viene effettuata specificando

server, database e credenziali, consentendo l'esecuzione diretta di istruzioni SQL dal codice Python. Una volta stabilita la connessione, viene creato un cursore che permette di eseguire operazioni di inserimento sulle tabelle del Data Warehouse.

Per garantire il rispetto dei vincoli di integrità referenziale, le tabelle vengono caricate seguendo un ordine prestabilito, coerente con le relazioni di *foreign key* definite nello schema:

- dimensioni indipendenti;
- dimensioni che dipendono da altre dimensioni;
- fact table.

In particolare, le dimensioni vengono caricate prima della fact table *FactParticipation*, assicurando che tutte le chiavi esterne siano già presenti nel database al momento dell'inserimento dei fatti.

Prima di avviare il caricamento, lo script verifica se una tabella è già popolata tramite una semplice query di conteggio. Se una tabella risulta non vuota, il caricamento viene saltato, evitando duplicazioni accidentali dei dati. Questo controllo rende lo script riutilizzabile e consente di rilanciare il processo senza dover ripulire manualmente il database.

I dati vengono letti dai file CSV prodotti nell'Assignment 5. Per ogni tabella, lo script:

- legge l'intestazione del file per determinare le colonne;
- costruisce dinamicamente la query di inserimento;
- effettua una conversione esplicita dei tipi di dato per alcuni attributi critici, in particolare:
 - *Streams1Month*, convertito in valore numerico;
 - *IsPrimary*, convertito in valore booleano/integer;
- gestisce correttamente i valori mancanti, convertendoli in NULL.

Poiché il caricamento completo dei dati può richiedere tempi significativi, l'inserimento delle tuple è stato progettato in modalità *batch*. I record vengono accumulati in gruppi di dimensione fissa e inseriti nel database tramite operazioni di inserimento multiplo, seguite da un commit esplicito.

Questa scelta consente di ridurre il numero di comunicazioni tra Python e il database, migliorare le prestazioni complessive del caricamento e rendere il processo più stabile in presenza di grandi volumi di dati.

Lo script presentato rappresenta la versione finale del processo di caricamento: durante lo sviluppo sono stati effettuati diversi test e lanci intermedi per verificare la connessione al database e il corretto inserimento dei dati, prima di consolidare la soluzione a batch adottata.

L'Assignment 6 conclude quindi il processo di costruzione del Data Warehouse, collegando la fase di preparazione dei dati alla loro effettiva persistenza nel database in modo automatico, coerente e ripetibile.

3.4 Assignment 8 – Numero di canzoni pubblicate per artista e anno

Il processo ha inizio con l'importazione della tabella *Fact Participation*. Successivamente, è stata applicata una trasformazione di tipo *Conditional Split* per selezionare esclusivamente le righe in cui l'artista risulta essere il contribuente principale.

Sono state quindi introdotte due trasformazioni di *Lookup*: la prima utilizzata per recuperare l'anno di pubblicazione dalla dimensione *Date*, la seconda per aggiungere il nome dell'artista dalla dimensione *Artist*.

Una trasformazione di tipo *Aggregate* ha consentito di raggruppare i dati per anno e artista, calcolando per ciascun caso il numero totale di brani pubblicati. Il risultato è stato successivamente ordinato per anno e, a parità di anno, per numero decrescente di canzoni, ottenendo la graduatoria richiesta.

Infine, il dataset risultante è stato esportato in un file *flat*, che viene utilizzato per la visualizzazione e il reporting finale.

3.5 Assignment 9 – Analisi Summer - Winter Score per regione

Il processo ha inizio con l'importazione della tabella *Fact Participation*, seguita da una trasformazione di tipo *Conditional Split* per selezionare esclusivamente le righe relative ai *Primary Artist*. Successivamente, sono state applicate tre trasformazioni di *Lookup*: la prima sulla dimensione *Artist*, utilizzata per ottenere il valore di *GeoKey*; la seconda sulla dimensione *ArtistGeography*, che consente di recuperare la regione e la nazione associate all'artista; la terza sulla dimensione *Date*, necessaria per determinare la stagione di pubblicazione del brano. In questo modo, ogni riga del flusso risulta caratterizzata sia dalla regione dell'artista sia dalla stagione associata al brano.

Prima di procedere all'aggregazione, è stata introdotta una trasformazione di tipo *Derived Column*, denominata *GroupRegion*, volta a riclassificare sotto un'unica etichetta (*Other*) tutti gli artisti privi di regione o non appartenenti all'Italia.

Il flusso è stato quindi suddiviso in due percorsi distinti, relativi rispettivamente ai brani pubblicati in estate e in inverno. Ciascun percorso è stato aggregato separatamente per *GroupRegion*, ottenendo, da un lato, il conteggio dei brani estivi e la somma degli *Streams1Month*, e, dall'altro, le corrispondenti misure relative ai brani invernali. Poiché la trasformazione *Merge Join* richiede flussi ordinati, entrambi i percorsi sono stati sottoposti a ordinamento prima di essere ricombinati. Nel *Merge Join* è stata utilizzata una *full outer join*, al fine di mantenere anche le regioni presenti in un solo flusso stagionale.

L'output risultante contiene, per ciascuna regione, quattro misure: il numero di brani estivi, il numero di brani invernali, la somma degli stream estivi e la somma degli stream invernali. Successivamente, una trasformazione di *Derived Column* ha consentito di calcolare i due indicatori richiesti: il *summer-winter score* basato sul numero di brani e quello basato sul numero di stream. Entrambi i calcoli includono una gestione esplicita dei casi in cui il denominatore risulti pari a zero, restituendo *NULL* e garantendo misure affidabili e facilmente interpretabili.

3.6 Assignment 10

L'esecuzione del flusso SSIS è stata avviata importando la tabella *Fact Participation*. Successivamente, sono state applicate tre trasformazioni di tipo *Lookup*: la prima sulla dimensione *Song*, utilizzata per recuperare l'attributo *Category*; la seconda sulla dimensione *Artist*, che consente di ottenere l'attributo *GeoKey*, necessario per il terzo *Lookup* sulla dimensione *ArtistGeography*, tramite il quale viene determinata la regione associata a ciascun artista.

Il flusso dei dati viene quindi duplicato mediante una trasformazione *Multicast*, al fine di eseguire due aggregazioni in parallelo: la prima calcola il numero di stream per ciascuna coppia (*Category*, *Region*), mentre la seconda determina il totale degli stream per ciascuna categoria, indipendentemente dalla regione.

I risultati delle due aggregazioni vengono successivamente ordinati e combinati tramite una trasformazione di tipo *Merge Join*, ottenendo per ciascuna riga sia il valore degli stream relativi a una specifica regione sia il totale degli stream della categoria corrispondente. Una trasformazione *Derived Column* consente quindi di calcolare il numero di stream nelle altre regioni e il rapporto richiesto dalla consegna, includendo una gestione esplicita dei casi in cui il denominatore risulti pari a zero.

Infine, prima della scrittura dell'output in formato *.txt*, le colonne *Category* e *Region* vengono convertite dal tipo Unicode (*DT_WSTR*) al tipo *DT_STR* mediante una trasformazione di *Data Conversion*, poiché la *Flat File Destination* non supporta tipi di dato Unicode.

3.7 Assignment 14 – Datacube

Per la creazione del cubo è stata configurata la connessione al database tramite la definizione del Data Source. Una volta stabilito il collegamento, abbiamo costruito il Data Source View (DSV), nel quale abbiamo importato la fact table e tutte le dimensioni rilevanti. Per ciascuna dimensione è stato selezionato un sottoinsieme di attributi, includendo esclusivamente quelli necessari per gli assignment successivi.

All'interno della dimensione Date è stata definita una gerarchia temporale, utilizzando gli attributi Year, Month e Day presenti nella dimensione. Per la definizione della gerarchia, sono stati introdotti due Named Calculations:

- **YearMonth** = concatenazione Year-Month nel formato YYYYMM
- **YearMonthDay** = concatenazione Year-Month-Day nel formato YYYYMMDD

Questi attributi ci hanno permesso di definire la gerarchia temporale:

$$\text{Year} \rightarrow \text{Month} \rightarrow \text{Day}$$

dove:

- il livello Month utilizza come NameColumn l'attributo **YearMonth**,
- il livello Day utilizza come NameColumn l'attributo **YearMonthDay**.

All'interno della dimensione Date era presente anche l'attributo Season, mantenuto come attributo flat perché non introduce una relazione di aggregazione naturale e stabile con gli altri livelli temporali (giorno, mese, trimestre, anno).

All'interno del cubo abbiamo definito una seconda gerarchia, appartenente alla dimensione Artist Geography, costruita per rappresentare il luogo di nascita degli artisti:

$$\text{Country} \rightarrow \text{Region} \rightarrow \text{Province} \rightarrow \text{BirthPlace}$$

Per ciascun livello di entrambe le gerarchie sono state mantenute le versioni sia l'attributo flat corrispondente.

Rispetto alla consegna del midterm, sono state apportate alcune modifiche al modello:

- eliminazione della misura **totalStreams**, in quanto ridondante rispetto alla misura **stream1Month**;
- eliminazione della misura calcolata **weightedStreams**, il cui calcolo viene eseguito dalla rispettiva query.

In particolare,

Inoltre, nel cubo è stata definita la dimensione **Role**, modellata come *degenerate dimension* per l'attributo **IsPrimary**. Tale attributo rappresenta una caratteristica intrinseca del fatto, priva di ulteriori attributi descrittivi o gerarchie.

3.8 Assignment 15 – Distribuzione mensile degli stream per regione e il totale della nazionale

L'obiettivo dell'analisi è la rappresentazione, su base mensile, di una struttura ordinata che elenca tutte le regioni appartenenti a ciascun paese, affiancata da una riga di sintesi per il totale nazionale e del relativo numero di stream osservati. Per rendere esplicita e leggibile tale struttura nel risultato finale, sono stati definiti due membri calcolati, *Paese_Col* e *Regione_Col*, che consentono di visualizzare le etichette testuali del paese e della regione nel set di risultato.

Questo approccio permette di gestire correttamente la riga del totale nazionale sostituendo la regione con una voce aggregata (*Total*), senza la necessità di creare nuovi elementi all'interno della gerarchia geografica del cubo.

La costruzione dell'insieme completo delle righe è realizzata mediante la funzione *GENERATE*, che per ciascun paese produce l'elenco delle regioni associate, ottenute tramite la funzione *DESCENDANTS* limitata al livello *Region*, e successivamente una riga aggiuntiva corrispondente al totale nazionale. La combinazione di tale struttura con il livello *Month* della gerarchia Date consente di replicare, per ogni mese, la sequenza *Paese-Regioni-Totale*.

La clausola *FILTER* è utilizzata per rimuovere i membri vuoti o privi di nome, mentre l'uso di *NON EMPTY* consente di escludere automaticamente le combinazioni per cui il numero di stream risulta assente. Infine, l'ordinamento del risultato è ottenuto concatenando mese, paese e regione, in modo da garantire una presentazione interpretabile dei dati.

3.9 Assignment 16 – Impatto annuale degli stream per artista

L'Assignment 16 ha l'obiettivo di calcolare l'impatto medio annuale degli stream per ciascun artista, introducendo una differenziazione in funzione del ruolo ricoperto (*Primary Artist* o *Featured Artist*) e normalizzando il risultato rispetto al totale degli stream osservati nello stesso anno, come richiesto dalla consegna.

- È stata definita una misura (*WeightedStreams*) che applica un coefficiente di ponderazione pari a 0.8 agli stream associati agli *Primary Artist* e pari a 0.2 nei casi in cui l'artista partecipi come *Featured Artist*, in accordo con le specifiche dell'Assignment.
- Per la normalizzazione, è stato definito un denominatore (*GrandTotalYear*) che rappresenta il **grand total degli stream nello stesso anno di riferimento**, calcolato utilizzando il membro *All* sulle dimensioni *Artist* e *Role*.
- La misura finale (*AvgYearlyArtistStreams*) è ottenuta come rapporto tra la somma degli stream pesati dell'artista e il totale degli stream dell'anno corrispondente. La definizione include controlli tramite le funzioni *IIF* e *IsEmpty*, al fine di prevenire divisioni per zero e garantire la robustezza e la pulizia del risultato finale.

È stata applicata la clausola *NON EMPTY* per limitare l'analisi alle sole combinazioni Artista-Anno effettivamente attive.

La percentuale ottenuta rappresenta una **media annuale normalizzata degli stream di ciascun artista**, calcolata come rapporto tra il contributo ponderato dell'artista e il grand total degli stream nello stesso anno.

3.10 Assignment 17 – Variazione percentuale annuale degli stream per categoria

L'Assignment 17 analizza la variazione percentuale annuale degli stream per ciascuna categoria musicale, confrontando il volume di stream registrato nell'anno corrente con quello osservato nello stesso periodo dell'anno precedente.

Per recuperare i valori relativi all'anno precedente è stata utilizzata la funzione *PARALLELPERIOD*, che consente di navigare la gerarchia temporale e di mantenere invariata la granularità temporale dell'analisi. Sulla base di tali valori, è stata definita una misura, *Percentage Variation*, che calcola la variazione percentuale degli stream tra due anni consecutivi. La definizione della misura include controlli tramite la funzione *IIF*, al fine di gestire i casi in cui il confronto non sia definito, come le divisioni per zero.

Nella formulazione iniziale, la query restituiva un numero elevato di valori *NULL* o non interpretabili, in particolare per il primo anno disponibile nel dataset, per il quale non esiste un periodo precedente di riferimento. Per migliorare la leggibilità e la qualità dell'output, è stato quindi applicato un filtraggio basato su *FILTER* e *NOT IsEmpty*, rimuovendo tutte le righe per cui il confronto temporale non risulta significativo.

Questo filtraggio consente di mantenere esclusivamente le osservazioni per le quali la variazione percentuale è effettivamente definita. In tale contesto, una variazione pari a -100% indica che una categoria musicale, pur essendo presente nell'anno precedente, non registra stream nell'anno corrente. In assenza di tale filtraggio, non sarebbe stato possibile distinguere tra una reale scomparsa di una categoria e una semplice assenza di dati storici.

Infine, l'ordinamento dei risultati sulla base della *Member_Key* della dimensione temporale garantisce una progressione cronologica coerente.

Variazioni percentuali positive segnalano la crescita o l'emergere di nuove categorie musicali, mentre variazioni negative evidenziano fasi di diminuzione. Un calo diffuso nell'ultimo anno disponibile può infine costituire un indicatore di incompletezza dei dati, poiché l'ultimo anno osservato non risulta ancora completo.

3.11 Assignment 18 – Confronto stagionale delle categorie rispetto alla media dell'anno precedente

L'Assignment 18 analizza le performance stagionali delle categorie musicali, confrontando il numero di stream osservato nell'anno corrente con la media della stessa stagione nell'anno precedente.

Per costruire il confronto, è stato definito un membro calcolato che determina la media stagionale degli stream nell'anno precedente. Questo valore è ottenuto utilizzando la funzione *PARALLELPERIOD* per risalire alla stessa stagione dell'anno precedente e la funzione *AVG* per calcolare la media degli stream su tutte le categorie. In questo contesto, la media stagionale è interpretata come una misura aggregata dell'intero mercato e non come una media storica specifica della singola categoria.

Sulla base di tale valore di riferimento, è stata calcolata la variazione percentuale degli stream di ciascuna categoria rispetto alla media stagionale dell'anno precedente. La definizione della misura include una gestione esplicita dei casi in cui il valore di riferimento risulti assente o pari a zero, al fine di evitare risultati non significativi o divisioni non valide.

Infine, è stato applicato un filtro per selezionare esclusivamente le categorie che presentano una performance superiore alla media della stagione precedente. Inoltre, sono state escluse la stagione *Unknown* e le combinazioni prive di dati, garantendo un output finale pulito e facilmente interpretabile.

L'output ottenuto consente di identificare le categorie musicali che mostrano una crescita significativa rispetto alla tendenza stagionale generale, fornendo indicazioni utili per l'analisi delle dinamiche di mercato.

3.12 Assignment 19 – Analisi stagionale e per categoria sull'impatto dei featuring

L'Assignment 19 estende l'analisi introdotta nell'Assignment 16, concentrandosi sull'impatto dei featuring in funzione della stagione e del genere musicale. L'obiettivo è valutare in quali

contesti stagionali e musicali la presenza di artisti in featuring contribuisca in modo significativo al volume di stream.

La query riprende la logica di ponderazione degli stream definita nell'Assignment 16, assegnando un peso pari a 0.8 agli stream associati all'artista principale e pari a 0.2 a quelli relativi agli artisti in featuring, applicando tale criterio a livello stagionale e di genere.

Dal punto di vista tecnico, l'analisi si basa sui seguenti accorgimenti:

- i membri *Unknown* della dimensione stagionale sono stati esclusi tramite la funzione *EXCEPT*, garantendo che l'analisi si basi esclusivamente su stagioni temporalmente definite;
- mediante la funzione *FILTER* sono stati selezionati solo i casi in cui l'indice calcolato risulta positivo, escludendo gli anni iniziali (1992–1995), nei quali il fenomeno dei featuring risulta assente.

L'introduzione del dettaglio stagionale consente di individuare con precisione il periodo in cui i featuring iniziano ad assumere un ruolo rilevante. I risultati mostrano che le collaborazioni diventano significative a partire dall'inverno del 1996; fino a tale momento, il mercato risulta prevalentemente caratterizzato da produzioni soliste.

Il confronto con i risultati dell'Assignment 16 evidenzia come l'impatto dei featuring non sia uniforme, ma fortemente dipendente dal contesto stagionale e dal genere musicale. In specifiche stagioni, generi come Trap e Rap raggiungono valori dell'indice di convenienza superiori al 20%, indicando che in tali ambiti il featuring rappresenta un fattore centrale nel successo stagionale.

Il valore massimo osservato nel 2025, pari al 33.33% per il genere *dark_poetic*, conferma il ruolo delle collaborazioni come componente rilevante nella determinazione delle performance stagionali di un genere musicale.

4 Experiments

Explain experimental setup, data, and evaluation.

5 Results

Present and discuss results.

6 Conclusion

Summarize findings and outline future work.